

Delete the Middle Node of a Linked List

Personal DSA Study Notes — Java

1. Introduction

Given the head of a singly linked list, delete its middle node and return the modified list. For an even-length list, the second middle node is deleted.

The key observation is that deletion requires the node before the node being deleted, because we must change its next reference.

2. Problem Statement

For an odd-length list there is one middle. For an even-length list there are two middle nodes, and the second middle is removed.

Input	Deleted	Result
1 → 2 → 3 → 4 → 5	3	1 → 2 → 4 → 5
1 → 2 → 3 → 4	3	1 → 2 → 4

3. Required Concepts

- Linked-list traversal using next.
- Integer division $n / 2$.
- Link manipulation: `previous.next = previous.next.next`.
- Slow/fast pointers.

4. Key Observation

We do not need to reach the middle node itself. We need the node immediately before it. Then the middle can be bypassed with:

```
previous.next = previous.next.next;
```

5. Approach 1 — Length + Second Traversal

First count the nodes. The required previous node is at position $n / 2$. Then traverse again to that position and skip its next node.

Pseudocode

```
if head == null or head.next == null:
    return null

temp = head
n = 0

while temp != null:
    n = n + 1
    temp = temp.next

position = n / 2
temp = head

while position > 1:
    temp = temp.next
    position = position - 1

temp.next = temp.next.next
return head
```

Original Core Algorithm

```
temp = head
n = 0
while temp != null:
    n++
```

```

        temp = temp.next

result = n / 2
temp = head
while temp != null:
    result--
    if result == 0:
        break
    temp = temp.next

temp.next = temp.next.next
return head

```

Complete Java Code

```

class Solution {
    public ListNode deleteMiddle(ListNode head) {
        if (head == null || head.next == null) {
            return null;
        }

        int n = 0;
        ListNode temp = head;

        // First pass: find length.
        while (temp != null) {
            n++;
            temp = temp.next;
        }

        int result = n / 2;
        temp = head;

        // Second pass: reach node before middle.
        while (temp != null) {
            result--;
            if (result == 0) {
                break;
            }
            temp = temp.next;
        }

        // Skip the middle node.
        temp.next = temp.next.next;

        return head;
    }
}

```

Static Visual

Before: 1 → 2 → 3 → 4 → 5 → NULL
 ↑ ↑
 previous middle

After: 1 → 2 ■■■■■ → 4 → 5 → NULL

Time: $O(n)$ overall. Extra space: $O(1)$. This is a two-pass solution.

6. Approach 2 — Adjusted Slow + Fast Pointers

The normal slow/fast technique finds the middle. Here we need the node before the middle, so the first slow movement is skipped. Fast moves two steps while slow moves one.

Pseudocode

```

if head == null or head.next == null:
    return null

slow = head
fast = head

fast = fast.next.next

while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next

```

```
slow.next = slow.next.next
return head
```

Original Core Algorithm

```
slow = head
fast = head
```

```
fast = fast.next.next
```

```
while fast != null AND fast.next != null:
    slow = slow.next
    fast = fast.next.next
```

```
slow.next = slow.next.next
return head
```

Complete Java Code

```
class Solution {
    public ListNode deleteMiddle(ListNode head) {
        if (head == null || head.next == null) {
            return null;
        }

        ListNode slow = head;
        ListNode fast = head;

        // Skip the first slow movement.
        fast = fast.next.next;

        // slow moves 1 step, fast moves 2.
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        // Bypass the middle node.
        slow.next = slow.next.next;

        return head;
    }
}
```

Static Visual

Odd: 1 → 2 → 3 → 4 → 5 → NULL
 ↑ ↑
 slow middle

Even: 1 → 2 → 3 → 4 → NULL
 ↑ ↑
 slow second middle

Deletion: slow.next = slow.next.next

Time: O(n) (one pass). Extra space: O(1).

7. Edge Cases

Case	Result
Empty list	null
One node	delete it → null
Two nodes	delete second node
Odd length	delete unique middle
Even length	delete second middle

8. Important Java Point

Use fast != null && fast.next != null in this order. Java evaluates && left-to-right, so fast must be checked before fast.next.

In Java, no explicit free/delete is needed for the removed node; once it is unreachable from the list, it can be garbage-collected.

9. Complexity Comparison

Approach	Time	Space	Passes
Length + traversal	$O(n)$	$O(1)$	2
Adjusted slow/fast	$O(n)$	$O(1)$	1

10. Quick Revision

1. Handle empty / one-node list.
2. Reach the node BEFORE the middle.
3. Use adjusted slow/fast for one pass.
4. Delete with:
 `slow.next = slow.next.next`
5. Return head.

Odd → delete unique middle
Even → delete second middle

11. Conclusion

The main trick is to reach the node before the middle. The two-pass method uses the list length; the optimized method adjusts the slow/fast pointer technique so slow stops one node before the node to delete.